PERSONAL QUANT SYSTEM — MASTER PROJECT PLAN

Production • Python/FastAPI • Supabase • AWS EC2 • ML • Backtesting • TradingView Paper Trading • Feedback Learning • AI Agent

Version 2.0 — October 2026

1. Tầm nhìn

Personal Quant System là hệ thống hỗ trợ ra quyết định đầu tư cá nhân, kết hợp Data Engineering, Machine Learning, Quantitative Research, Backtesting và Forward Testing. Hệ thống tự động thu thập dữ liệu, lưu vào Supabase, tạo feature, dự đoán, sinh tín hiệu, đánh giá rủi ro và theo dõi kết quả giao dịch mô phỏng. TradingView Paper Trading là môi trường forward-testing; kết quả được đưa ngược vào hệ thống để phân tích và tạo feedback dataset cho các vòng huấn luyện tiếp theo. Hệ thống không tự động đặt lệnh bằng tiền thật.

```
MARKET DATA
    ↓
Data Collection → Validation → Supabase
    ↓
Feature Engineering → ML Model
    ↓
Risk + Signal → Backtesting
    ↓
TradingView Paper Trading
    ↓
Trade Outcome → Supabase
    ↓
Feedback Dataset → Error Analysis
    ↓
Retraining → New Model → Forward Testing → LOOP
```

2. Mục tiêu cuối cùng

- Tự động thu thập dữ liệu thị trường theo ngày và chạy pipeline khoảng 23:00 giờ Việt Nam.
- Supabase PostgreSQL là database production.
- FastAPI là backend Python chạy trên AWS EC2.
- Data validation, retry, logging, monitoring và notification.
- Feature engineering + ML prediction có model version.
- Backtesting có transaction cost, slippage và chống look-ahead bias.
- Risk management, position sizing và BUY/HOLD/SELL signal.
- TradingView Paper Trading để forward-test.
- Thu thập entry, exit, P/L, holding period và metrics của trade.
- Biến feature + prediction + trade outcome thành feedback dataset.
- Retraining định kỳ, đánh giá out-of-sample và chỉ thay model khi đạt tiêu chí.
- React dashboard, CI/CD, monitoring, model lifecycle và AI Agent documentation.

3. Technology Stack

Tầng	Công nghệ	Vai trò
Backend	Python + FastAPI	API và orchestration
Database	Supabase PostgreSQL	Production data
ORM	SQLAlchemy	Database access
Migration	Alembic	Schema versioning
Validation	Pydantic	API/config/data validation
Data	pandas + NumPy	Processing
ML	scikit-learn	Baseline/classical ML
Advanced ML	XGBoost / LightGBM	Model candidates
Testing	pytest	Automated tests
Quality	Ruff + mypy	Lint/type checking
Package	uv + pyproject.toml	Python environment
Container	Docker	Deployment
Cloud	AWS EC2	Backend + jobs
Proxy	Nginx	HTTPS/reverse proxy
Frontend	React + Vite	Dashboard
Forward test	TradingView Paper Trading	Simulated trading
CI/CD	GitHub Actions	Test/build/deploy
Scheduler	APScheduler / AWS scheduler	Daily automation

4. Repository Structure

```
personal-quant-system/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── core/
│   │   ├── api/v1/
│   │   ├── schemas/
│   │   ├── models/
│   │   ├── repositories/
│   │   ├── services/
│   │   └── jobs/
│   ├── tests/
│   ├── pyproject.toml
│   ├── uv.lock
│   └── Dockerfile
├── ml/
│   ├── training/
│   ├── evaluation/
│   ├── experiments/
│   └── models/
├── backtesting/
│   ├── engine.py
│   ├── strategy.py
│   ├── metrics.py
│   └── costs.py
├── frontend/
├── database/migrations/
├── docs/
│   ├── architecture/
│   ├── data/
│   ├── ml/
│   ├── quant/
│   ├── tradingview/
│   └── infrastructure/
```

```
├── agent/
│   ├── AGENTS.md
│   ├── PROJECT_RULES.md
│   ├── CODING_RULES.md
│   ├── ARCHITECTURE_RULES.md
│   ├── WORKFLOW.md
│   └── TASKS.md
├── .github/workflows/
├── docker-compose.yml
└── README.md
```

5. Backend Architecture

```
API Router → Pydantic Schema → Service → Repository → Supabase
                      ↑
                Business logic
                nằm ở Service
```

- Router không chứa SQL/ML/business logic.
- Repository chỉ xử lý database.
- Job scheduler gọi pipeline/service, không copy business logic.
- Production config dùng environment variables.
- Mọi schema change dùng migration.

6. Database Design

Bảng	Mục đích
stocks	Danh sách mã, exchange, sector, active
market_prices	OHLCV theo symbol/date
technical_indicators	SMA, EMA, RSI, MACD, ATR, volatility
features	Feature snapshot cho ML
predictions	Prediction + probability + model_version
signals	BUY/HOLD/SELL + confidence/risk
trade_plans	Entry/SL/TP/quantity
paper_trades	Actual entry/exit/fees/P&L
trade_metrics	Return, holding period, MFE, MAE
backtest_runs	Backtest performance
portfolio	Paper portfolio + P/L
pipeline_logs	Job status/errors
model_versions	Model lifecycle/metrics
feedback_dataset	Feature + prediction + outcome + lineage

market_prices nên unique theo (stock_id, date). Prediction → signal → trade phải truy ngược được qua signal_id/trade_id/model_version.

7. Data Pipeline & Validation

```
External Data → Collector → Normalizer → Validator → Upsert Supabase
                                                        ↓
                                  Indicators → Features → ML → Signal
```

- Kiểm tra missing/duplicate.
- Kiểm tra OHLC consistency.
- Kiểm tra volume và price anomalies.

- Kiểm tra timestamp/timezone và data freshness.
- Không đưa dữ liệu chưa validate vào training/prediction.

8. Daily Automation — 23:00

```
23:00
 ↓
Load active symbols
 ↓
Fetch latest available data
 ↓
Retry failures
 ↓
Validate
 ↓
Upsert Supabase
 ↓
Indicators + Features
 ↓
Run current ML
 ↓
Risk + Signal + Trade Plan
 ↓
Save results + pipeline log
 ↓
Notification
 ↓
END
```

Job phải chống chạy trùng và kiểm tra dữ liệu đã sẵn sàng; 23:00 là thời điểm mục tiêu theo giờ Việt Nam.

9. Machine Learning Plan

- Baseline: Logistic Regression.
- Tiếp theo: Random Forest.
- XGBoost/LightGBM khi pipeline ổn định.
- Không bắt đầu bằng deep learning.
- Không random shuffle financial time series.
- Không dùng future information.
- Đánh giá bằng cả ML metrics và trading metrics.

10. Feature Engineering

- Return 1D/5D/20D.
- SMA20/50/200, EMA12/26.
- RSI, MACD, ROC.
- ATR, volatility.
- Volume change/ratio.
- Price/SMA ratios.
- Market/sector context.
- Feature timestamp phải chỉ dùng thông tin đã có tại prediction time.

11. Target & Validation

- Next-day return direction.
- Next-5-day return.
- Threshold-based future return.
- So sánh nhiều target.

```
Walk-forward:
Train 2018–2022 → Validation 2023 → Test 2024
Train 2019–2023 → Validation 2024 → Test 2025
```

Giữ holdout/out-of-sample. Không tune trên test. Label được phép dùng future outcome cho training, nhưng future-derived information tuyệt đối không được xuất hiện trong input tại prediction time.

12. Backtesting

```
Historical Data → Feature Snapshot → Prediction → Strategy Rule
→ Signal → Position Sizing → Cost + Slippage → Portfolio → Metrics
```

- Total/annual return.
- Sharpe/Sortino.
- Maximum drawdown.
- Win rate/profit factor.
- Transaction cost/slippage.
- Benchmark comparison.
- No look-ahead bias.

13. Risk Management & Signal

- Risk per trade.
- Position sizing.
- Portfolio/sector exposure.
- Volatility filter.
- Stop loss/take profit.
- Maximum drawdown.
- Correlation/concentration.

```
ML probability + Technical context + Market context + Risk + Exposure
                          ↓
                BUY / HOLD / SELL
                          ↓
            Entry / SL / TP / Quantity
```

14. TradingView Paper Trading

TradingView là lớp forward-testing. Personal Quant vẫn là trung tâm của data/ML/backtesting. Người dùng có thể lấy trade plan từ hệ thống và thực hiện giao dịch mô phỏng trên TradingView.

```
Personal Quant → Prediction → Signal → Trade Plan
                                  ↓
                    TradingView Paper Trading
                                  ↓
```

```
Entry → Exit → P/L
          ↓
  Personal Quant
```

15. Feedback Learning — điểm mở rộng quan trọng nhất

```
Model v1
 ↓
Prediction + Features
 ↓
Paper Trade
 ↓
Actual Entry / Exit / P&L
 ↓
Feedback Dataset
 ↓
Error + Performance Analysis
 ↓
Retraining Candidate
 ↓
Out-of-Sample Evaluation
 ↓
Deploy only if better
 ↓
Model v2
 ↓
Forward Test
 ↺
```

Không train trực tiếp sau từng trade. Ban đầu dùng batch retraining; sau khi ổn định mới cân nhắc weekly/monthly retraining. Online learning chưa phải ưu tiên.

16. Feedback Dataset

Nhóm	Dữ liệu
Time	feature_timestamp, prediction_timestamp, entry_time, exit_time
Identity	symbol, signal_id, trade_id, model_version
Features	RSI, SMA, returns, volume, volatility, context...
Prediction	probability, predicted_class, confidence
Decision	signal, entry, SL, TP, quantity
Execution	actual_entry, actual_exit, fees, slippage
Outcome	return_pct, pnl, holding_period, WIN/LOSS
Path	MFE, MAE
Lineage	dataset_version, feature_version, source

17. Ví dụ

```
FPT — 2026-10-02
RSI=62 | Price/SMA50=1.032 | VolumeRatio=1.40 | Volatility=0.021
Model v1.0 → P(UP)=0.78 → BUY
Plan: Entry 95,000 | SL 91,000 | TP 103,000 | Qty 100

Paper Trade:
Actual Entry = 95,200
Actual Exit  = 102,000
```

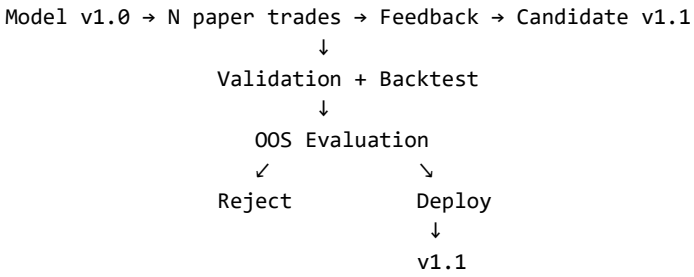
```
Outcome      ≈ +7.14%
P/L before fees ≈ +680,000 VND
Result       = WIN

→ lưu feedback
→ phân tích theo confidence/model/regime
→ dùng cho vòng retraining sau
```

18. Quy tắc chống Data Leakage

1. Không đưa future price/return vào feature tại prediction time.
2. Không random split time series.
3. Không dùng test set để tune model.
4. Không dùng P/L sau trade làm input cho chính prediction đã tạo trade.
5. Không để feedback của một trade quay ngược thời gian.
6. Luôn lưu timestamp của feature, prediction, entry, exit.
7. Luôn lưu model_version và feature_version.
8. Không thay model chỉ vì vài trade thắng.
9. Candidate model phải qua OOS evaluation trước production.

19. Model Versioning



Mọi prediction/trade phải truy ngược được về model version.

20. Performance Analysis

- Directional accuracy.
- Precision/Recall/F1/ROC-AUC khi phù hợp.
- Probability calibration.
- Win rate theo confidence bucket.
- P/L theo signal, market regime, volatility, sector.
- MFE/MAE.
- Holding period.
- Drawdown.
- Sharpe/Sortino.
- So sánh model versions.

21. Portfolio

- Cash.

- Holdings.
- Average price.
- Market value.
- Realized/unrealized P/L.
- Allocation/exposure.
- Historical equity.
- Paper portfolio tách biệt khỏi real-money portfolio.

22. API Plan

```
GET  /health
GET  /api/v1/stocks
GET  /api/v1/stocks/{symbol}/prices
GET  /api/v1/predictions
GET  /api/v1/signals
GET  /api/v1/trade-plans
GET  /api/v1/paper-trades
GET  /api/v1/portfolio
POST /api/v1/backtests
GET  /api/v1/backtests/{id}
GET  /api/v1/models
GET  /api/v1/feedback/metrics
GET  /api/v1/pipeline/status
```

23. Frontend Dashboard

- Dashboard: market, portfolio, signals, pipeline.
- Stock Detail: chart, indicators, prediction.
- Signals: BUY/HOLD/SELL.
- Paper Trades: entry/exit/P&L.
- Model: versions + metrics.
- Feedback: performance theo confidence/regime/model.
- Backtest: equity curve/Sharpe/drawdown.
- Portfolio: allocation/P&L.

24. AWS + Docker + CI/CD

```
AWS EC2
 ├── Ubuntu
 ├── Docker
 │    ├── FastAPI
 │    └── Daily Worker/Scheduler
 └── Nginx → HTTPS
          ↓
       Supabase

git push → GitHub Actions → tests → lint/typecheck
→ Docker build → registry → EC2 deploy → health check
```

- Secrets không commit.
- Supabase là database production managed.
- EC2 có thể chạy backend + scheduler.

- Docker là deployment unit.
- Health check sau deploy.

25. Monitoring

- Health endpoint.
- Pipeline duration/status.
- Rows fetched/inserted/updated.
- Failed symbols.
- Prediction/signal count.
- Paper trade import/sync status.
- Errors.
- Notification khi job fail.

26. AI Agent Documentation

```
agent/
├── AGENTS.md
├── PROJECT_RULES.md
├── CODING_RULES.md
├── ARCHITECTURE_RULES.md
├── WORKFLOW.md
└── TASKS.md

docs/
├── ARCHITECTURE.md
├── DATABASE.md
├── API.md
├── DATA_PIPELINE.md
├── ML.md
├── BACKTESTING.md
├── TRADINGVIEW.md
├── FEEDBACK_LEARNING.md
├── RISK_MANAGEMENT.md
├── DEPLOYMENT.md
└── TESTING.md
```

26.1 AGENTS.md

```
AI Agent MUST:
1. Read AGENTS.md.
2. Read relevant docs.
3. Inspect existing code.
4. Propose a small plan.
5. Implement only the requested scope.
6. Run tests/lint/type checks.
7. Review diff.
8. Update docs when architecture changes.

Rules:
API → Service → Repository → DB
No secrets in Git.
No schema changes without migration.
No data leakage.
No unrelated refactor.
Preserve model/data lineage.
```

26.2 TASK Template

```
# TASK-001
Title: Implement market price repository
Status: TODO

Requirements:
- Insert
- Upsert
- Query by symbol/date
- Prevent duplicates

Acceptance Criteria:
- Tests pass
- Type hints
- Error handling
- No unrelated changes
```

27. Roadmap theo Phase

Phase	Nội dung	Milestone
0	Architecture + AI Agent docs	Bộ luật + tài liệu
1	Python setup	FastAPI + uv + tooling
2	Supabase	Schema + migrations
3	Data Collector	Fetch data
4	Validation	Data sạch
5	Daily Pipeline	Pipeline local
6	FastAPI	API
7	Testing	pytest/Ruff/mypy
8	Docker	Container
9	AWS EC2	Production
10	Scheduler	23:00
11	Feature Engineering	Feature pipeline
12	ML v1	Baseline
13	Walk-forward	Validation
14	Backtesting	Quant engine
15	Risk	Risk engine
16	Signal	BUY/HOLD/SELL
17	TradingView	Paper/forward test
18	Trade Results	Entry/exit/P&L
19	Feedback Dataset	Feature+prediction+outcome
20	Error Analysis	Model failure analysis
21	Retraining	Candidate models
22	OOS Evaluation	Model selection
23	Portfolio	Paper portfolio
24	Frontend	Dashboard
25	Notification	Reports
26	Monitoring	Health/logs
27	CI/CD	Deployment
28	Model Versioning	Lifecycle
29	AI Assistant	Explainability/research

28. Milestone thực tế

```
A. Data Foundation
Supabase + Collector + Validation
```

```
        ↓
B. Automation
Daily Pipeline + Tests + Docker
        ↓
C. Production
EC2 + Nginx + 23:00
        ↓
D. Quant Engine
Features + ML + Backtest + Risk
        ↓
E. Forward Testing
TradingView Paper Trading
        ↓
F. Feedback Learning
Trade results → Feedback → Retraining
        ↓
G. Decision Support
Portfolio + Dashboard + Monitoring
        ↓
H. Mature System
CI/CD + Model Versioning + AI Assistant
```

29. Definition of Done

- Daily data tự động cập nhật.
- Không duplicate dữ liệu.
- Pipeline có retry/logging.
- FastAPI production ổn định.
- EC2 + Docker + Nginx hoạt động.
- 23:00 automation hoạt động.
- ML reproducible.
- Backtest không leakage.
- Risk + Signal hoạt động.
- TradingView paper results có thể ghi nhận.
- Feedback dataset có lineage.
- Retraining có validation/OOS.
- Dashboard theo dõi lifecycle.
- AI Agent docs đủ để Agent mới hiểu architecture.
- Secrets không nằm trong Git.

30. Thứ tự triển khai từ project hiện tại

1. Tạo /docs + /agent
2. Viết AGENTS.md + ARCHITECTURE.md
3. Setup FastAPI + uv + pyproject
4. Thiết kế Supabase schema
5. Chuyển data storage sang Supabase
6. Data Collector + Validation
7. daily_pipeline.py
8. Docker
9. AWS EC2 + Nginx
10. Scheduler 23:00
11. Tích hợp ML hiện tại
12. Backtesting

13. Risk + Signal
14. Trade Plan
15. TradingView Paper Trading
16. Thu thập Entry/Exit/P&L
17. Feedback Dataset
18. Error Analysis
19. Retraining Candidate
20. OOS Evaluation
21. Model Versioning
22. Dashboard + Monitoring
23. CI/CD + AI Assistant

31. Kết luận

Personal Quant System phiên bản mục tiêu không chỉ là một model dự đoán giá. Nó là một closed-loop quantitative research and decision-support system: dữ liệu → ML → signal → backtest → TradingView forward test → kết quả mô phỏng → feedback dataset → retraining → model mới → forward test. Supabase là trung tâm dữ liệu; FastAPI là backend; AWS EC2 là production runtime; TradingView là forward-testing layer; AI Agent là công cụ triển khai theo bộ luật Markdown.

Ưu tiên hiện tại: xây Data Foundation và Automation thật chắc trước khi tối ưu ML. Sau khi pipeline production ổn định mới mở rộng Feedback Learning.